CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Technology and Engineering — DEPSTAR

ITUE301 — Advanced Web Development Frameworks

B.Tech. – CSE, CE, IT | Semester: 5th | AY 2026–27

OPEN-BOOK PRACTICAL EXAMINATION

SET A — Campus Canteen Pre-Order

Duration / Marks	2 Hours	20 Marks (5 tasks × 4 marks)
Tech Stack	React (Vite) + Express.js + MongoDB (Mongoose)
Syllabus	Practical 1 to 5 only	Open-book: internet and AI tools allowed

Instructions

- AI tools and the internet are allowed. You will be asked to explain your work in a short viva.
- Keep a short FINDINGS.md file in the root of your repository. For **each of Tasks 1–5**, create a clearly labelled section (T1, T2, T3, T4, T5). In each section, write in your own words:
    - **What was wrong:** Identify the problem or incomplete functionality you found in the starter code.
    - **Why it mattered:** Explain the effect of the problem or the incorrect behaviour it caused.
    - **How you fixed it:** Describe the changes you made and mention the relevant files or functions. Ensure that your explanation accurately describes your implementation.
- Create a **PUBLIC GitHub repository** named itue301-exam-[id]-[batch]. Complete your work and submit it with **ONE final commit only.** Do not commit .env files containing secrets or node_modules.
- After completing and checking all five tasks, copy your repository URL and obtain the final commit SHA. Put **BOTH the public repository URL and the final commit SHA in a single file named as [id]-[set].pdf and submit it** to the designated **Google Classroom assignment** before the examination ends. Ensure the repository is public and the SHA matches your final commit.

The Project

The campus canteen lets students browse the menu, build an order and read the day's specials. Staff place and manage orders through an API, and menu data is stored in MongoDB. Every menu item has: name, category (snack, meal or drink), price in ₹, an availability flag, calories per 100 g (kcal) and an optional portion weight in grams (100 g when not given).

A junior developer has left the project in the starter kit. The modules below do not yet meet the rules stated for them. For each task, understand the rules, find what stops the module from meeting them, and make it meet them. The tasks are independent.

Task 1 — The Menu (Components and Props) 4 Marks

- Each card shows: name, category, price as ₹ with two decimals, portion weight in g, energy in kcal, and the label “Available” or “Not Available” (from the availability flag).
- Energy (kcal) = calories per 100 g × portion weight ÷ 100, rounded to the nearest whole number. Portion weight is weightG, or 100 when weightG is absent.
- The page raises no React warnings in the browser console.

Task 2 — The Order (Routing and State) 4 Marks

- Menu, Order and Specials open without a full-page reload.
- The name typed in the box is shown as “Order for: <name>”.
- Only available items can be added. Each cart line shows item name, price (₹, two decimals) and energy (kcal, as in Task 1), and can be removed.
- The page shows the item count, the total price (₹, two decimals) and the total energy (kcal). All three always equal the sums over the cart lines and are not kept in separate state.
- An order holds at most 1000 kcal. An item that would take the total above 1000 kcal is not added, and the message “Calorie limit of 1000 kcal exceeded” is shown.

Task 3 — The Specials Board (API Consumption) 4 Marks

- On opening, request https://jsonplaceholder.typicode.com/todos?_limit=10 exactly once. Show “Loading specials…” until it finishes.
- Each todo is a special: show its title and “Sold out” if completed is true, otherwise “Available”. List available specials first; otherwise keep the API order.
- Show “<n> of <total> available”, where n is the number of specials that are not sold out.
- If the request fails (network error, or HTTP status outside 200–299) show “Could not load specials” and a Retry button that requests again without reloading the page. No exception may go unhandled.

Task 4 — The Orders Service (Express and Middleware) 4 Marks

- Menu (kcal per 100 g): Samosa 260, Masala Chai 50, Veg Thali 150, Cold Coffee 70.
- POST /api/v1/orders takes { itemName, qty, weightG }. itemName must be a menu item; qty is a whole number from 1 to 10 (default 1); weightG is a number above 0 (default 100). Otherwise respond 400 { "error": "..." }. On success respond 201 with { id, itemName, qty, weightG, kcal }, where kcal = round(kcal per 100 g × weightG ÷ 100 × qty).
- GET /api/v1/orders responds 200 with the list. GET and DELETE /api/v1/orders/:id respond 200 when the order exists, otherwise 404 { "error": "..." }.
- Every request logs one line: [METHOD] [PATH] [TIMESTAMP]. Any unhandled error (GET /api/v1/boom throws on purpose) responds 500 { "error": "Something went wrong" } with no stack trace in the response.

Task 5 — The Menu Data (MongoDB and Mongoose) 4 Marks

- name: required, trimmed, at most 50 characters. price: required, 0 or more. category: snack, meal or drink, default snack. available: default true.
- caloriesPer100g: required, 0 to 900. weightG: default 100, from 1 to 2000; the weightG of a drink may not exceed 500.
- formatValidationError(err) returns the plain object { error: "Validation failed", details: [{ field, message }, …] }.
- seed.js saves one valid item and prints the formatted error for one invalid item. Save a screenshot of the saved document in Compass/Atlas as evidence/db.png.